UNIVERSIDAD NACIONAL DE LA PLATA

FACULTAD DE INGENIERÍA e INFORMÁTICA



Sistemas Distribuidos y Paralelos

Trabajo Final

INTEGRANTES: 02275/2 del Blanco, Javier Alberto
02530/5 Pereira, Armando Antonio

- 2026 -

ÍNDICE

1. Descripción del algoritmo secuencial **1**
2. Estrategia y etapas de diseño paralelo **2**
3. Tiempos de ejecución, métricas y análisis de escalabilidad **3**
Speedup 4
Eficiencia 5
Balance de carga 5
Escalabilidad 6
Ley de Amdahl 7
Ley de Gustafson-Barsis 7
Análisis de Escalabilidad 7
Modelo de isoeficiencia 8
4. Uso de herramientas de inteligencia artificial **8**
5. Conclusiones **10**

1. Descripción del algoritmo secuencial

El problema de las N-Reinas consiste en ubicar N reinas en un tablero de N×N de manera que ninguna pueda atacarse entre sí. En el ajedrez, las reinas se atacan si comparten la misma fila, columna o cualquier diagonal. El objetivo es contar la cantidad total de disposiciones válidas, considerando también las simetrías del tablero para obtener soluciones únicas.

La representación del tablero en el algoritmo secuencial de referencia utiliza una técnica optimizada basada en bits, conocida como bitboard. Cada fila del tablero se representa mediante un número entero donde cada bit encendido indica la presencia de una reina en esa columna. Para un tablero de N reinas, se necesitan N bits, que caben cómodamente en una variable entera estándar. Esta representación permite realizar operaciones de verificación de ataques mediante operaciones lógicas bit a bit, que son extremadamente rápidas.

El algoritmo mantiene tres máscaras que representan las columnas atacadas desde la posición actual. La máscara down registra las columnas ocupadas por reinas en filas anteriores, indicando ataques verticales. La máscara left rastrea las diagonales que descienden hacia la izquierda, desplazándose una posición hacia la izquierda en cada fila. La máscara right hace lo propio con las diagonales hacia la derecha, desplazándose hacia la derecha. La unión de estas tres máscaras, mediante una operación OR, produce el conjunto completo de columnas que están siendo atacadas. Para determinar las columnas seguras, se toma la máscara global que tiene todos los bits en 1 y se le resta, mediante una operación AND con el complemento, el conjunto de columnas atacadas. El resultado es una máscara donde cada bit encendido representa una columna donde puede colocarse la siguiente reina sin generar conflictos.

La estrategia para determinar si un tablero es válido se basa en un proceso de backtracking que coloca reinas fila por fila. En cada paso, el algoritmo calcula las columnas seguras utilizando las máscaras de ataques, extrae el bit menos significativo de esa máscara mediante una técnica aritmética conocida como aislar el bit menos significativo, coloca una reina en esa columna y actualiza las máscaras de ataques para la siguiente fila. Este proceso se repite recursivamente hasta completar todas las filas o hasta que no queden columnas seguras. Cuando se logra colocar la última reina, se ha encontrado una solución válida.

Una optimización fundamental del algoritmo consiste en explotar las simetrías del tablero para evitar explorar configuraciones redundantes. El algoritmo divide la búsqueda en dos fases. En la primera fase (Backtrack1), coloca la primera reina en la esquina superior izquierda y la segunda reina únicamente en las columnas centrales, evitando los bordes por simetría. En la segunda fase (Backtrack2), coloca la primera reina en las columnas del centro del tablero y explora simultáneamente posiciones simétricas mediante dos índices que avanzan desde los extremos hacia el centro. Esta estrategia reduce drásticamente el espacio de búsqueda.

Los resultados se almacenan en tres contadores que clasifican las soluciones según sus simetrías. Las soluciones que no presentan ninguna simetría, es decir, que al aplicarles rotaciones y reflexiones generan ocho configuraciones distintas, se acumulan en un contador denominado COUNT8. Las soluciones que presentan simetría de orden 4, lo que significa que son iguales a sí mismas al rotarlas 180 grados generando cuatro configuraciones distintas, se acumulan en COUNT4. Las soluciones que presentan simetría de orden 2, iguales a sí mismas al rotarlas 90 grados generando dos configuraciones, se acumulan en COUNT2. Al finalizar la búsqueda, el número de soluciones únicas se obtiene sumando los tres contadores, mientras que el número total de disposiciones contando todas las simetrías se obtiene multiplicando cada contador por la cantidad de configuraciones que genera ese tipo de simetría y sumando los resultados.

2. Estrategia y etapas de diseño paralelo

El diseño de la solución paralela sigue las cuatro etapas propuestas por Ian Foster: descomposición, comunicación, aglomeración y mapeo. La arquitectura objetivo es híbrida, combinando memoria distribuida entre nodos de un cluster y memoria compartida dentro de cada nodo multicore.

En la etapa de descomposición, se identifica que las tareas pueden definirse basadas en los datos de entrada, en este caso por la posición de la primera reina en el tablero. El algoritmo secuencial ya realiza esta descomposición natural al explorar diferentes valores de BOUND1 para el Backtrack1 y pares de BOUND1 y BOUND2 para el Backtrack2. Cada una de estas configuraciones iniciales genera una tarea independiente que puede resolverse mediante backtracking sin dependencias con las demás tareas. Estas tareas son estáticas, ya que se conocen a priori cuántas son antes de comenzar la ejecución, y presentan un volumen de trabajo no uniforme, pues algunas posiciones iniciales generan más soluciones que otras. Para mitigar este desbalance, se implementó una estrategia que avanza dos niveles extra en la recursividad inicial. Esto representa que cada tarea enviada a los hilos ya no es un tablero vacío, sino un escenario parcialmente resuelto donde las reinas de las primeras filas ya han sido posicionadas en casillas válidas. Esta estrategia fue pensada para los valores de N elegidos para este trabajo, para valores más pequeños o más grandes sera necesario cambiar los niveles de recursividad inicial.

En la etapa de comunicación, se determinan los patrones necesarios para coordinar el trabajo entre los nodos del cluster. Dado que las tareas son independientes, la comunicación requerida es mínima y se limita a la distribución inicial de trabajo y la recolección de resultados. Se adopta un patrón de comunicación master-worker, donde un proceso master posee un hilo principal que envía bloques de tareas, que se encuentran depositadas en un arreglo en el proceso master, al otro hilo principal del proceso worker, que lo descompone y genera su propio arreglo de tareas para que demás hilos puedan acceder a estas. A su vez, los hilos del master realizan tareas accediendo a estas directamente al arreglo de tareas principal, sin pasar por el hilo principal del máster. El hilo principal del master asume tareas solo cuando no tiene peticiones pendientes, mientras que el hilo principal del worker hace lo propio mientras existan tareas locales y no se requiera solicitar más carga al nodo central. Finalmente todos los hilos que realizaron tareas

envían sus resultados al hilo principal del master que hará el cálculo final de las soluciones. Para esto, worker y el master suman todos las soluciones locales a ellos y se realiza un MPI_Reduce para juntar los valores de ambos procesos en el master. Este patrón es local, estructurado, estático, lectura-escritura y asíncrono. La comunicación es unidireccional desde el hilo principal del master hacía el hilo principal worker para el envío de tareas, y desde el hilo principal del worker hacia el hilo principal del máster para la devolución de resultados.

Dentro de cada nodo, donde se utiliza memoria compartida, la comunicación entre hilos se realiza a través de una estructura de datos compartida. Se utiliza una cola de tareas protegida por mecanismos de sincronización, donde los hilos trabajadores toman tareas para procesar. Este patrón de comunicación también es local, ya que los hilos acceden a la misma memoria, y estructurado, pues sigue un protocolo definido de toma de tareas y acumulación de resultados.

En la etapa de aglomeración, en lugar de enviar cada tarea individualmente desde el master al worker, estas se agrupan en bloques. El tamaño del chunk se define mediante una constante que puede ajustarse. Para determinar el tamaño del bloque de tareas, se analizó un escenario crítico que podría derivar en la subutilización de recursos (específicamente con N=14 y 16 hilos). Bajo estas condiciones, el sistema genera aproximadamente 3000 tareas; con el fin de equilibrar la carga de trabajo entre todos los hilos, se estimó que cada chunk debería contener al menos 187.5 tareas (3000/16). Con base en este cálculo, se optó por establecer el valor en 256, por ser la potencia de 2 más próxima. Esta estrategia reduce la frecuencia de las comunicaciones MPI, que son costosas en comparación con el tiempo de cómputo, y mejora la localidad de los datos dentro de cada proceso. También se decide que el proceso maestro participe activamente del cómputo, creando sus propios hilos de cómputo para procesar tareas locales, en lugar de permanecer inactivo mientras distribuye trabajo.

En la etapa de mapeo, se realiza un mapeo dinámico, ya que los bloques de tareas se asignan bajo demanda al worker cuando este los pida. Además, dentro de cada nodo, se aplica un mapeo dinámico para la distribución de tareas entre los hilos, utilizando un arreglo compartido (para el caso del master, este arreglo es el arreglo de tareas principal). Cuando un hilo queda ocioso, toma la siguiente tarea disponible del arreglo. Esta estrategia centralizada, aunque introduce cierta contención por el acceso al mutex de la cola, logra un buen balance de carga debido a que el volumen de trabajo entre tareas no es uniforme.

3. Tiempos de ejecución, métricas y análisis de escalabilidad

Para la evaluación de la solución paralela, se consideran las configuraciones requeridas por la cátedra: 4 unidades de procesamiento distribuidas como 2 cores por nodo, 8 unidades como 4 cores por nodo, y 16 unidades como 8 cores por nodo. Las cargas de trabajo a evaluar son tableros de 14, 15, 16, 17 y 18 reinas. Para poder realizar la medición del speedup y la eficiencia fue necesario medir los tiempos de ejecución tanto del algoritmo secuencial como el de nuestras solución con mpi + pthreads como se en la tabla 1:

Tabla 1: Tabla de tiempos de ejecución

	TAMAÑO DEL PROBLEMA (N)				
UNIDADES DE PROCESAMIENTO	14	15	16	17	18
SECUENCIAL	0,181344	1,113602	7,256482	50,168613	364,493008
2 maquinas, 2 hilos en cada una	0.097647	0.457960	2.311690	18.219178	123.951316
2 maquinas, 4 hilos en cada una	0.052757	0.239754	1.181589	8.775696	61.769253
2 maquinas, 8 hilos en cada una	0.028244	0.128534	0.614294	4.161100	31.541152

Speedup

El speedup se calcula como el cociente entre el tiempo de ejecución del mejor algoritmo secuencial disponible para el mismo tamaño de problema y el tiempo de ejecución del algoritmo paralelo. Para nuestra implementación los resultados fueron los siguientes (tabla 2):

Tabla 2: Tabla de SpeedUP

	TAMAÑO DEL PROBLEMA (N)				
UNIDADES DE PROCESAMIENTO	14	15	16	17	18
SECUENCIAL	1	1	1	1	1
2 maquinas, 2 hilos en cada una	1.86	2.43	3.14	2.75	2.94
2 maquinas, 4 hilos en cada una	3.44	4.64	6.14	5.72	5.90
2 maquinas, 8 hilos en cada una	6.42	8.66	11.81	12.06	11.56

Como se puede apreciar en la tabla 2 todos los speedup fueron inferiores a la cantidad de unidades de procesamiento, por lo que el speedup fue sublineal para todos los casos, lo cual es lógico debido a características (overhead) inherentes al algoritmo paralelo. Se observa además speedup crecientes al aumentar al aumentar la cantidad de hilos por nodo, con crecimientos cada vez menores. Además, si bien entre N = 16 y N = 17 se nota una caída en los speedup, en general se observan crecimientos en los speedup al aumentar la carga de trabajo.

Eficiencia

La eficiencia se obtiene dividiendo el speedup por la cantidad total de unidades de procesamiento utilizadas. Nuestros resultados fueron (tabla 3):

Tabla 3: Tabla de Eficiencia

	TAMAÑO DEL PROBLEMA (N)				
UNIDADES DE PROCESAMIENTO	14	15	16	17	18
SECUENCIAL	1,00	1,00	1,00	1,00	1,00
2 maquinas, 2 hilos en cada una	0.465	0.6079	0.7848	0.6884	0.7352
2 maquinas, 4 hilos en cada una	0.4297	0.5806	0.7677	0.7146	0.7376
2 maquinas, 8 hilos en cada una	0.4013	0.5415	0.7383	0.7535	0.7223

En la tabla 3 se puede observar eficiencias menores a 1 y crecientes tanto al aumentar la cantidad de nodos, como al aumentar la cantidad de trabajo. Nuevamente, se aprecia una anomalía al pasar de N = 16 a N = 17.

Balance de carga

El balance mide qué tan equitativamente se distribuye el trabajo entre los hilos o procesos en un algoritmo paralelo. Se calcula haciendo el promedio de tiempo trabajo de los hilos sobre el tiempo de trabajo del hilo que más trabajo. Nuestras mediciones dieron los siguientes resultados (tabla 4, tabla 5 y tabla 6):

Tabla 4: Tabla de Balance de carga para 2 hilos por nodo

2 máquinas, 2 hilos en cada una					
TAMAÑO DEL PROBLEMA (N)	14	15	16	17	18
Promedio	0.060176	0.394215	2.217057	16.469320	115.391740
Máx	0.063034	0.457960	2.311690	18.219178	123.951316
Balance	0.9547	0.8608	0.9591	0.9040	0.9309

Tabla 5: Tabla de Balance de carga para 4 hilos por nodo

2 máquinas, 4 hilos en cada una					
TAMAÑO DEL PROBLEMA (N)	14	15	16	17	18
Promedio	0.038198	0.188417	1.101552	7.727319	55.560636
Máx	0.052756	0.239754	1.181589	8.775696	61.769253
Balance	0.7241	0.7859	0.9323	0.8805	0.8995

Tabla 6: Tabla de Balance de carga para 8 hilos por nodo

2 máquinas, 8 hilos en cada una					
TAMAÑO DEL PROBLEMA (N)	14	15	16	17	18
Promedio	0.021056	0.094816	0.558146	3.825318	27.944968
Máx	0.028244	0.128534	0.614294	4.161100	31.541152
Balance	0.7455	0.7377	0.9086	0.9193	0.8860

A partir del análisis de las tablas 4, 5 y 6, se observa que la configuración de 2 hilos por máquina presenta el balance más alto y estable (promedio superior a 0.92), seguida por la de 4 hilos (0.84) y la de 8 hilos (0.83). El balance se mantiene notablemente sólido para todas las configuraciones en problemas grandes (N≥16), superando el 0.88 incluso con 8 hilos por nodo.

Esto indica que el modelo híbrido gestiona con éxito la escalabilidad del sistema. Aunque el balance se reduce levemente en problemas pequeños (N=14,15) debido al overhead de sincronización y gestión, el sistema logra una distribución de tareas muy equitativa para cargas pesadas. Esto permite que el incremento en el paralelismo se traduzca en una mejora directa del tiempo total de ejecución sin sacrificar la eficiencia del cluster.

Escalabilidad

Indica la habilidad para reaccionar y adaptarse sin perder calidad, manejar el crecimiento continuo de trabajo y estar preparado para escalar sin perder calidad en los servicios ofrecidos. Para sacar conclusiones respecto a ella debemos analizar los siguientes puntos:

Ley de Amdahl

Para una determinada carga de trabajo, a menos que prácticamente todo el programa sea paralelizable, el aumento de rendimiento será muy limitado, independientemente del número de unidades de procesamiento disponibles

De acuerdo a lo observado en la tabla de speedup, a medida que se aumenta la cantidad de hilos para una misma carga de trabajo (N constante), el speedup tiende a disminuir, por lo cual se concluye que la ley de Amdahl se cumple.

Ley de Gustafson-Barsis

Si el tamaño del problema aumenta a medida que se incrementa
la cantidad de unidades de procesamiento, el impacto de la fracción secuencial del programa se vuelve cada vez menos importante

De acuerdo a lo observado en la tabla de speedup, incrementar la carga de trabajo (aumentando N) para una misma cantidad de hilos genera que el speedup aumente (exceptuando al pasar N = 16 a N = 17), por lo cual se comprobó que la ley de Gustafson-Barsis se cumple.

Análisis de Escalabilidad

La escalabilidad de un algoritmo puede categorizarse de la siguiente manera:

Escalabilidad Débil: si la eficiencia se mantiene aproximadamente constante al incrementar simultáneamente el número de unidades de
procesamiento y el tamaño del problema

Escalabilidad Fuerte: si la eficiencia se
mantiene aproximadamente constante al incrementar el número de unidades de procesamiento sin aumentar el tamaño del problema

Escalabilidad Nula: si la eficiencia disminuye significativamente al aumentar la cantidad de unidades de procesamiento, independientemente de si se incrementa o no el tamaño del problema

El algoritmo posee escalabilidad débil ya que al aumentar la cantidad de nodos y la cantidad de carga, se observa crecimiento en la eficiencia. Además, posee

escalabilidad escalabilidad fuerte, ya que al aumentar la cantidad de nodos y mantener la carga de trabajo constante la eficiencia se mantiene estable.

4. Uso de herramientas de inteligencia artificial

En el desarrollo de este trabajo se utilizaron asistentes de inteligencia artificial como herramientas de apoyo en el proceso de diseño e implementación. Las herramientas empleadas fueron DeepSeek y ChatGPT, cada una con roles complementarios en diferentes etapas del desarrollo.

DeepSeek se utilizó principalmente para el análisis y comprensión del algoritmo secuencial de referencia. A partir de descripciones del problema y fragmentos del código original, la herramienta ayudó a identificar las tareas paralelizables, comprender la representación bitboard y las optimizaciones por simetría implementadas por Takaken. También fue útil para el diseño de la estrategia de particionamiento de tareas, sugiriendo la división por primera reina como punto natural de paralelismo.

ChatGPT se empleó para la generación de código y la resolución de problemas específicos de implementación. Por ejemplo, se utilizó para generar la estructura base de un programa híbrido MPI con Pthreads, para corregir errores de compilación relacionados con tipos de datos en las funciones MPI_Send y MPI_Recv, y para implementar la cola de tareas con mutex y variables de condición. En varias ocasiones fue necesario repreguntar y ajustar los prompts para obtener el código deseado.

La siguiente tabla resume las interacciones más significativas:

Tabla 7: Tabla de consultas a herramienta ChatGPT.

Herramienta: ChatGPT		
Prompt	**Éxito**	**Observación**
Explicar el algoritmo secuencial de N-Reinas de takaken, identificando las funciones Backtrack1, Backtrack2 y Check.	SÍ	Identificó correctamente que Backtrack2 es la función más costosa (80% del tiempo) y Check valida rotaciones.
Generar código MPI básico para que el root distribuya tareas a workers usando comunicación punto a punto.	Parcial	El código usaba broadcast y no manejaba workers inactivos. Se requirió mejorar con MPI_Probe.
Agregar array thread_times[32] con mutex para medir tiempo individual de cada hilo Pthread.	SÍ	Implementó local_thread_times estático + pthread_mutex_t time_mutex.

¿Cómo implementar MPI_Probe con MPI_ANY_SOURCE y MPI_ANY_TAG para recibir peticiones dinámicas?	SÍ	Generó root_main_thread con detección de TAG_PETICION vs TAG_RESULT.

Tabla 8: Tabla de consultas a herramienta DeepSeek.

Herramienta: DeepSeek		
Prompt	**Éxito**	**Observación**
Diseñar estrategia de partición del espacio de soluciones usando BOUND1 para Backtrack1 y BOUND1/BOUND2 para Backtrack2.	SÍ	Propuso generar tareas desde task_generator() con dos bucles: uno para Backtrack1 y otro para Backtrack2.
¿Es mejor enviar tareas una por una o en bloques (chunks) para reducir latencia de comunicación?	Sí	Recomendó CHUNK_SIZE=8 como equilibrio entre latencia y granularidad.
Implementar worker_pthreads_manager con cola de tareas usando pthread_cond_wait y pthread_cond_signal.	SÍ	Entregó código productor-consumidor con tasks_consumed y manager_cond.

Tabla 9: Tabla de consultas a herramienta Claude.

Herramienta: Claude		
Prompt	**Éxito**	**Observación**
Diseñar estructura de cola de tareas compartida con Task struct conteniendo type, bound1, bound2, endbit, lastmask.	SÍ	Definió Task con todos los campos necesarios para Backtrack1 y Backtrack2.
Implementar f_thread_root y f_thread_worker que consumen tareas de task_queue con mutex.	Sí	Entregó hilos que toman tareas secuencialmente protegiendo next_task_id y tasks_consumed.
Eliminar variables globales innecesarias. Unificar thread_times_root y thread_times_worker en un solo array local.	Parcial	Propuso struct TimeInfo, pero se optó por solución más simple (array en main).
Agregar medición de tiempo	No	Código innecesario.

del manager MPI separada del tiempo de cómputo de los hilos.		Repreguntamos.

5. Conclusiones

Si bien se logró una solución paralela y distribuida del algoritmo de n-reinas y esta disminuye drásticamente los tiempos de ejecución con respecto a la solución secuencial a medida que aumenta la carga de trabajo, esta no es escalable ya que a partir de 4 hilos por nodo baja su eficiencia. A futuro podría probarse con más cantidad de hilos o en otro cluster para estudiar si este comportamiento es consistente y persiste.